Voici une trame que tu peux quasiment réutiliser telle quelle dans ta présentation, avec un **workflow clair** et un **schéma d'architecture textuel** pour tes encadrants.

1. Vision globale : ce que fait ton PFE

Message clé à faire passer en 1 slide :

> Mon PFE construit une **brique data RAG-ready** pour les repos publics STM32Cube (H7, H5, …) :
>
> - Ingestion des issues GitHub et fichiers de documentation.
> - Nettoyage et normalisation.
> - Enrichissement par heuristiques (board, component, layer, severity, etc.).
> - Chunking RAG-ready.
> - Scripts d'analyse / évaluation (statistiques, TF-IDF, similarité).
> - Un agent IA d'ingénierie (Copilot) pour analyser et documenter.

Tu ne construis pas le chatbot final, mais **tout le pipeline data qui le rend possible**.

2. Workflow complet (étape par étape)

Tu peux structurer ta démo autour de ce pipeline :

1. **[1] Ingestion**
2. **[2] Cleaning (nettoyage / normalisation)**
3. **[3] Prétraitement V2 (enrichissement, heuristiques)**
4. **[4] Similarité entre issues**
5. **[5] Chunking RAG-ready**
6. **[6] Tests & évaluation (statistiques, TF-IDF)**
7. **[7] Agent IA de support (Copilot / Nova)**

Je détaille chaque étape avec ce que tu peux dire en réunion.

3. Schéma d'architecture globale (à mettre en slide)

Tu peux mettre ce genre de schéma "boîte à boîtes" (tu le refais proprement sur PowerPoint / draw.io) :

Text

```
        +-----------------+
        | GitHub Public   |
        | STM32CubeH7/H5  |
        +--------+--------+
                 |
                 v
        [1] Ingestion (scripts fetch_*)
        - fetch_issues.py
        - fetch_files.py
                 |
                 v
        data/raw_issues_*.json
        data/raw_files_*.json
                 |
                 v
        [2] Cleaning (clean_*)
        - clean_issues.py
        - clean_files.py
                 |
                 v
        data/clean_issues_*.json
        data/clean_files_*.json
                 |
                 v
        [3] Prétraitement V2 (docs_*_v2)
        - issues_to_docs_v2.py
        - files_to_docs_v2.py
        (PréprocessingModel + heuristiques)
                 |
                 v
        data/docs_issues_*_v2.json
        data/docs_files_*_v2.json
                 |
                 +-----------------------+
                 |                       |
                 v                       v
     [4] Similarité issues        [5] Chunking RAG-ready
  - compute_issue_similarity_v2.py  - docs_to_chunks_issues_v2.py
                                    - docs_to_chunks_files_v2.py
                 |                       |
                 v                       v
     data/docs_issues_*_v2_sim.json   data/chunks_issues_*_v2.json
                                      data/chunks_files_*_v2.json
                 |
                 v
        [6] Tests & Evaluation
        - test_stats_*.py
        - test_tfidf_search_*.py
        - test_consistency_*.py

                 |
                 v
        [7] Agent IA d'analyse (VS Code)
        - .github/agents/
        - .github/skills/
        (aide à l'analyse & documentation)
```

Et pour le **PC hors domaine**, tu ajoutes à droite :

Text

```
      data/chunks_*_v2.json  --->  Indexation vectorielle (Chroma / FAISS)
                                  + RAG sandbox (embeddings + retrieval)
```

4. Détail par étape – ce que tu expliques à l'oral

4.1. [1] Ingestion – récupération des données GitHub

Objectif : récupérer toutes les données brutes nécessaires depuis les repos STM32Cube.

- **Issues GitHub**
  - Script : `src/scripts/fetch_issues.py`
  - Service : `src/services/github_service.py`
  - Ce que tu dis :
    - Je me connecte à l'API GitHub pour un repo (ex. `STM32CubeH7`).
    - Je récupère toutes les issues (open + closed), en **excluant les PR**.
    - Je stocke les issues brutes dans `data/raw_issues_<repo>.json`.
- **Fichiers de documentation**
  - Script : `src/scripts/fetch_files.py`
  - Service : `src/services/file_collect_service.py`
  - Ce que tu dis :
    - Je parcours le repo cloné localement (`GitHub_repos/STM32CubeH7`).
    - Je collecte :
      - `README.md` à la racine,
      - les README sous `Projects/**/README*`,
      - les `Release_Notes*`,
      - les README de `Documentation/`.
    - Je stocke tout dans `data/raw_files_<repo>.json`.

Message clé : à la fin de cette étape, j'ai **toutes les données brutes** (issues + docs) dans `data/raw_*.json`.

4.2. [2] Cleaning – normalisation et structuration du texte

Objectif : rendre les données **propres, homogènes et structurées**.

- **Issues**
  - Script : `clean_issues.py`
  - Service : `cleaning_service.py`
  - Actions :
    - Normalisation des sauts de ligne.
    - Concaténation : `body + commentaires`.
    - Remplacement des images par une balise textuelle `[IMAGE ATTACHED]` et conservation de `image_urls`.
    - Construction d'un champ **`clean_text` structuré** :

      Text

      ```
      Title: ...
      Body:
      ...
      Comments:
      [author - date]
      ...
      ```
  - Sortie : `data/clean_issues_<repo>.json`.
- **Fichiers**
  - Script : `clean_files.py`
  - Actions :
    - Normalisation du contenu (`\r\n` → `\n`, `strip`).
    - Détection d'un champ `file_type` à partir du chemin :
      - `root_readme`, `project_readme`, `release_notes`, `doc_readme`, `other`.
  - Sortie : `data/clean_files_<repo>.json`.

Message clé : ici, je ne fais encore **aucune compréhension métier**, juste du **nettoyage** pour avoir un texte propre.

4.3. [3] Prétraitement V2 – enrichissement sémantique

Objectif : transformer chaque issue/fichier nettoyé en **document riche**, prêt pour le RAG.

4.3.1. Issues – `PreprocessingModelIssuesV2`

- Script : `issues_to_docs_v2.py`
- Modèle : `PreprocessingModelIssuesV2` dans `preprocessing_model.py`
- Heuristiques : `stm32cube_preprocessing_heuristics.py`

À partir de `clean_issues_<repo>.json`, tu produis des documents du type :

Json

```json
{
  "id": "stm32cubeh7-issue-338",
  "source_kind": "issue",
  "repo": "STM32CubeH7",
  "mcu_package": "STM32CubeH7",
  "mcu_series": "H7",
  "type": "issue",
  "path": null,
  "github_url": "https://github.com/STMicroelectronics/STM32CubeH7/issues/338",
  "board": "STM32H743VITX",
  "category": null,
  "component": "ADC",
  "example_name": null,
  "issue_number": 338,
  "issue_title": "...",
  "state": "open",
  "labels": ["bug"],
  "is_confirmed_bug": true,
  "created_at": "...",
  "closed_at": null,
  "ingest_version": "v2.0",
  "clean_text": "Title: ... Body: ...",
  "image_urls": [],
  "images_count": 0,
  "is_valid": true,
  "layer": "CubeMX",
  "severity": "medium",
  "issue_kind": "bug_report"
}
```

Ce que tu expliques comme heuristiques clés :

- **`is_valid`**
  - `false` si issue trop courte ou labels `invalid`, `duplicate`, `wontfix`.
  - Sinon `true`.
  - But : filtrer le bruit avant indexation.
- **`issue_kind`**
  - Valeurs : `bug_report`, `usage_question`, `configuration_help`, `other`.
  - Basé sur mots-clés dans le titre/texte.
- **`layer`**
  - `HAL`, `LL`, `Middleware`, `CubeMX`, `Other`.
  - Permet de savoir à quel niveau de la stack STM32Cube se situe le problème.
- **`severity`**
  - `high` si présence de mots forts (hardfault, crash, hang, stuck, corruption…),
  - `low` si typo/doc,
  - sinon `medium`.
- **`board`**
  - Détection de références cartes (ex. `NUCLEO-H743ZI`, `STM32H743VITX`).
- **`component`**
  - `ETH`, `USB`, `ADC`, `DMA`, `UART`, `CAN`, `FREERTOS`, etc.
  - Heuristique améliorée pour ne pas confondre `CAN` (périphérique) avec le verbe anglais "can".

Sortie : `data/docs_issues_<repo>_v2.json`.

4.3.2. Fichiers – `PreprocessingModelFilesV2`

- Script : `files_to_docs_v2.py`
- Entrée : `clean_files_<repo>.json`
- Exemple de doc :

Json

```json
{
  "id": "stm32cubeh7-file-6606534015442931394",
  "source_kind": "file",
  "repo": "STM32CubeH7",
  "mcu_package": "STM32CubeH7",
  "mcu_series": "H7",
  "type": "file",
  "path": "README.md",
  "file_type": "root_readme",
  "github_url": null,
  "board": "NUCLEO-H7A3ZI",
  "category": null,
  "component": "USB",
  "example_name": null,
  "ingest_version": "v2.0",
  "clean_text": "...",
  "is_valid": true
}
```

Heuristiques :

- `file_type` : `root_readme`, `project_readme`, `release_notes`, `doc_readme`, `other`.
- `board` : détectée dans le path ou le texte.
- `component` : ex. `USB`, `ETH`, etc.
- `example_name` : ex. `ETH_BasicTxRx` à partir du path `Projects/NUCLEO-H743ZI/Examples/ETH/ETH_BasicTxRx/README.md`.

Sortie : `data/docs_files_<repo>_v2.json`.

Message clé : à cette étape, chaque issue / document devient un **document structuré riche en metadata métier**, parfaitement exploitable par un moteur RAG.

4.4. [4] Similarité entre issues – `V2_sim`

Objectif : pouvoir proposer des **issues similaires** (pour un agent de support ou RAG).

- Script : `compute_issue_similarity_v2.py`
- Techno : `scikit-learn` (TF-IDF + `cosine_similarity`)

Pipeline :

1. Lecture de `docs_issues_<repo>_v2.json`.
2. Filtrage sur `is_valid=true`.
3. Construction d'un vecteur TF-IDF sur `issue_title + clean_text`.
4. Pour chaque issue, calcul des plus proches voisines.
5. Ajout d'un champ `related_issue_ids` (liste d'IDs proches).

Sortie :
`data/docs_issues_<repo>_v2_sim.json` (même schema que V2 + `related_issue_ids`).

Message clé : c'est une première couche "intelligente" sans LLM, pour enrichir encore les metadata.

4.5. [5] Chunking – préparation RAG-ready

Objectif : transformer chaque document V2 en **petits chunks de texte** avec metadata, prêts à être indexés dans une base vectorielle.

- Fichier utilitaire : `src/services/chunk_service.py`
  - `split_by_paragraphs(text, max_chars)` :
    - Découpe un texte en blocs de paragraphes de taille ≤ `max_chars`.
  - `doc_issue_to_chunks(doc, max_chars)` :
    - 1 issue V2 → 1..N chunks (texte + metadata).
  - `doc_file_to_chunks(doc, max_chars)` :
    - 1 fichier V2 → 1..N chunks (texte + metadata).

Chunking issues

- Script : `docs_to_chunks_issues_v2.py`
- Entrée : `docs_issues_<repo>_v2_sim.json` (ou `_v2.json`).
- Traitement :
  - Ignore `is_valid=false`.
  - Utilise `doc_issue_to_chunks(doc, max_chars=1200)`.
- Sortie : `data/chunks_issues_<repo>_v2.json`.

Chunking fichiers

- Script : `docs_to_chunks_files_v2.py`
- Entrée : `docs_files_<repo>_v2.json`.
- Traitement :
  - Ignore `is_valid=false`.
  - Utilise `doc_file_to_chunks(doc, max_chars=1200)`.
- Sortie : `data/chunks_files_<repo>_v2.json`.

Structure d'un chunk (idée) :

- `text` : bloc de texte (un ou plusieurs paragraphes).
- `metadata` :
  - Issues : `issue_number`, `github_url`, `layer`, `severity`, `issue_kind`, `board`, `component`, `related_issue_ids`, etc.
  - Fichiers : `file_type`, `path`, `board`, `component`, `example_name`, etc.

Message clé : ces JSON de chunks sont **exactement** ce qu'on voudra indexer dans un Chroma/FAISS pour un RAG.

4.6. [6] Tests & évaluation – valider la qualité du pipeline

Objectif : vérifier que le prétraitement V2 est cohérent, riche et utile pour un RAG.

Scripts principaux :

- `test_stats_issues_v2.py`
  - Statistiques sur : `layer`, `severity`, `issue_kind`, `boards`, `components`.
- `test_stats_files_v2.py`
  - Stats sur : `file_type`, `boards`, `components`, `is_valid`.
- `test_consistency_issues_v2_vs_sim.py`
  - Vérifie que `docs_issues_v2_sim` a les mêmes colonnes que `docs_issues_v2` (sauf `related_issue_ids`).
- `test_tfidf_search_issues_v2.py`
  - Recherche TF-IDF sur les issues (sans LLM) pour tester la pertinence du contenu prétraité.
- `test_tfidf_search_files_v2.py`
  - Recherche TF-IDF sur les fichiers.

Ce que tu expliques :

- Ces tests servent à :
  - Vérifier que les heuristiques fonctionnent correctement (par ex. réduire le biais `component=CAN`).
  - Produire des **stats exploitables** pour montrer à ST que le modèle "comprend" la structure du code et des bugs.
  - Démontrer que les docs/chunks V2 sont **exploitable pour un RAG** avant même l'indexation vectorielle.

4.7. [7] Agent IA Copilot (VS Code) – outil d'ingénierie

Objectif : se servir de l'IA comme copilote pour analyser et documenter ton pipeline.

- Fichiers :

  Text

  ```
  .github/
    agents/
      STM32CubePreprocessing.agent.md
    skills/
      analyzestats/SKILL.md
      document_json_schema/SKILL.md
      chunking_strategies/SKILL.md (optionnel)
    instructions/
      STM32CubePreprocessing.md
    contracts/
      stm32cube_preprocessing.contract.md
  ```
- Rôle :
  - **`analyzestats`** : interpréter les stats générées par tes scripts.
  - **`document_json_schema`** : générer la documentation des schémas JSON (docs_*v2, chunks*_v2).
  - **`chunking_strategies`** : t'aider à rédiger la section "bonnes pratiques de chunking / RAG".

Message clé :
L'agent n'est **pas** dans le pipeline d'exécution, c'est un **outil d'ingénieur** pour mieux analyser, documenter et expliquer ton travail.

5. État d'avancement (à résumer en fin de présentation)

Tu peux terminer avec un slide "Etat d'avancement" :

- ☑ Ingestion issues + fichiers (H7, H5)
- ☑ Cleaning et normalisation (issues + fichiers)
- ☑ Prétraitement V2 (issues + fichiers) avec heuristiques métier (layer, component, board, severity, issue_kind, …)
- ☑ Similarité issues (TF-IDF) → `docs_issues_*_v2_sim.json`
- ☑ Chunking RAG-ready (issues + fichiers) → `chunks_*_v2.json`
- ☑ Scripts de stats, cohérence, TF-IDF search
- ☑ Agent IA Copilot pour analyse & documentation
- 🔜 Sur PC hors domaine :
  - Indexation des `chunks_*_v2.json` dans un moteur vectoriel (Chroma / FAISS),
  - Sandbox RAG (embeddings + retrieval + éventuellement génération),
  - Évaluation de différentes stratégies de chunking,
  - Rédaction de la partie "évaluation & bonnes pratiques RAG" du mémoire.

Si tu veux, tu peux m'envoyer un **brouillon de slide** ou te concentrer sur une étape (par exemple la partie chunking ou les heuristiques V2), et je te prépare le texte exact à mettre dans ta présentation (titre + bullet points).